

基于OpenCV的SIFT特征检测

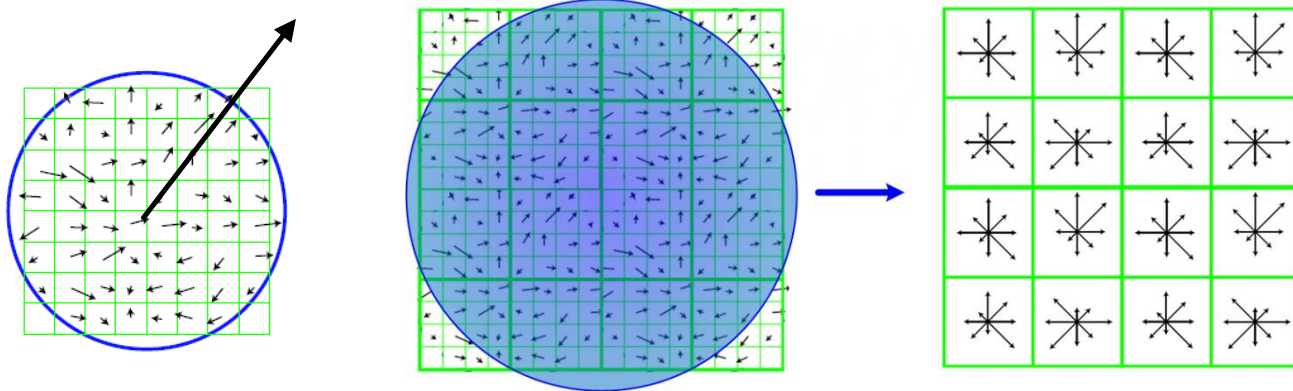
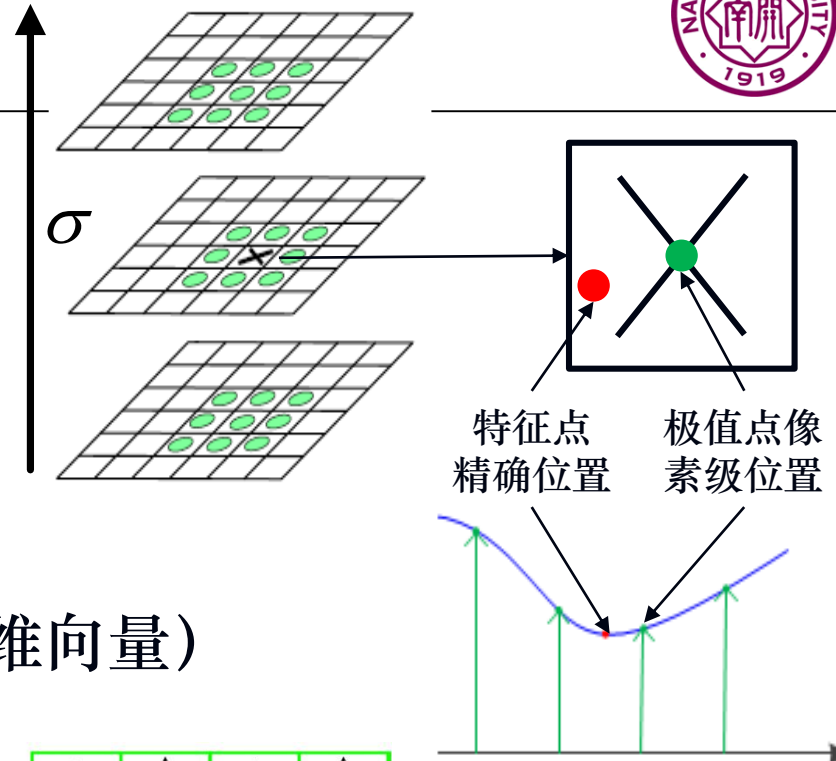
SITF based on OpenCV



SIFT特征检测

➤ SIFT特征检测的步骤

1. 检测尺度空间的极值点
2. 精确定位特征点
3. 设定特征点的方向参数
4. 生成特征点的描述子（128维向量）



SIFT特征检测源码

来源: OpenCV2.4.9



SIFTDetect ()

1. createInitialImage ()
 2. SIFT::buildGaussianPyramid ()
 3. SIFT::buildDoGPyramid ()
 4. SIFT::findScaleSpaceExtrema
 5. {
 6. for each layer, row, column
 7. find local extrema with pixel accuracy
 8. adjustLocalExtrema ()
 9. if isLocalExtrema
 10. calcOrientationHist ()
 11. }
 12. calcDescriptors ()
- 1) 构建图像金字塔
- 2) 检测极值点
- 3) 精确定位特征点
- 4) 设定特征点主方向
- 5) 生成特征描述子

SIFT特征检测源码——构造函数

```
CV_WRAP explicit SIFT( int nfeatures=0, int nOctaveLayers=3,  
    double contrastThreshold=0.04, double edgeThreshold=10,  
    double sigma=1.6);
```

- nfeatures: 设定检测出的特征点数，默认值为0，如果nfeatures不为0，则算法对检出的特征点排名，返回最好的nfeatures个特征点
- nOctaveLayers: DoG金字塔中，金字塔每组可检测特征的层数，默认值为3
- contrastThreshold: 去除不稳定极值点时，使用的阈值。contrastThreshold越大，返回的特征点数越少
- edgeThreshold: 去除边缘响应过大的极值点时，使用的阈值。edgeThreshold越大，返回的特征点数越多（被多滤掉的越少）
- sigma: 高斯金字塔第0层的高斯滤波系数

SIFT特征检测源码——检测函数

```
void operator()(InputArray img, InputArray mask,  
                vector<KeyPoint>& keypoints,  
                OutputArray descriptors,  
                bool useProvidedKeypoints=false) const;
```

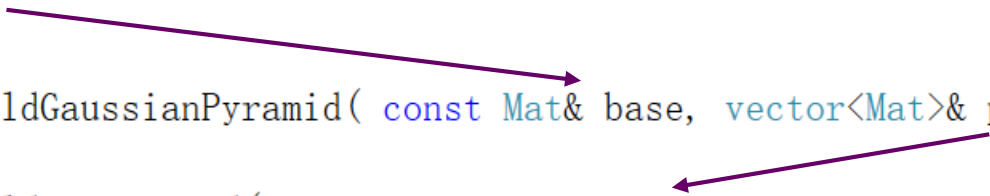


```
void SIFT::myDetect(InputArray image,  
                    vector<KeyPoint>& keypoints,  
                    OutputArray descriptors,  
                    bool useProvidedKeypoints = false) const;
```

- img: 输入图像
- mask: 图像检测区域（可选）
- keypoints: 保存特征点的向量
- descriptors: 保存特征点描述子的矩阵
- useProvidedKeypoints: 是否使用提供的特征点，默认为false，在程序中检测特征点

1) 构建图像金字塔

```
static Mat createInitialImage( const Mat& img, bool doubleImageSize, float sigma )  
void buildGaussianPyramid( const Mat& base, vector<Mat>& pyr, int nOctaves ) const;  
void buildDoGPyramid( const vector<Mat>& pyr, vector<Mat>& dogpyr ) const;
```



- createInitialImage: 全局函数，根据参数获得金字塔底层数据
 - SIFT::buildGaussianPyramid : SIFT成员函数，计算高斯金字塔
 - SIFT::buildDoGPyramid: SIFT成员函数，计算高斯差分金字塔
-
- doubleImageSize: 金字塔底层是否将原图像增倍，默认为true
 - nOctaves: 图像金字塔组数，与图像大小有关

```
cvRound(log( (double)std::min( base.cols, base.rows ) ) / log(2.) - 2) - firstOctave
```

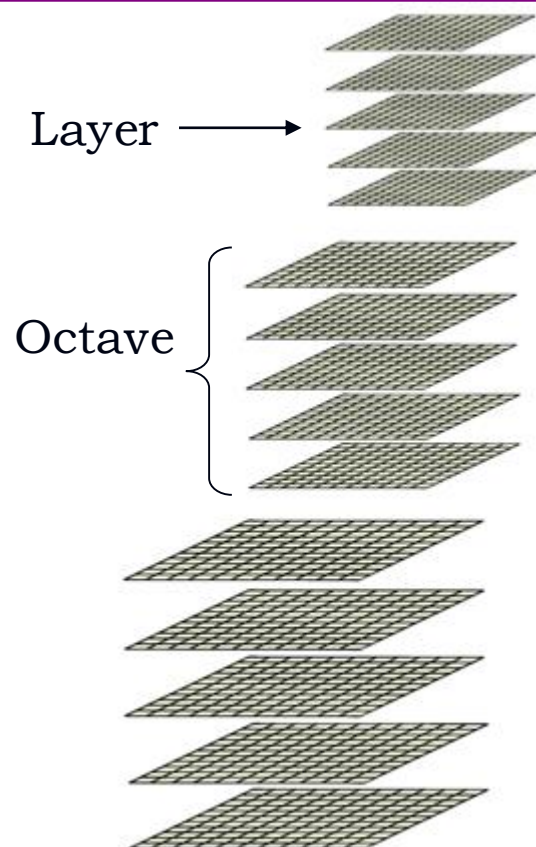
1) 构建图像金字塔

```
static Mat createInitialImage( const Mat& img, bool doubleImageSize, int nOctaves)  
void buildGaussianPyramid( const Mat& base, vector<Mat>& pyr,  
void buildDoGPyramid( const vector<Mat>& pyr, vector<Mat>& dog
```

- createInitialImage: 全局函数，根据参数获得金字塔底层图像
- SIFT::buildGaussianPyramid : SIFT成员函数，构建高斯金字塔
- SIFT::buildDoGPyramid: SIFT成员函数，计算DOG金字塔

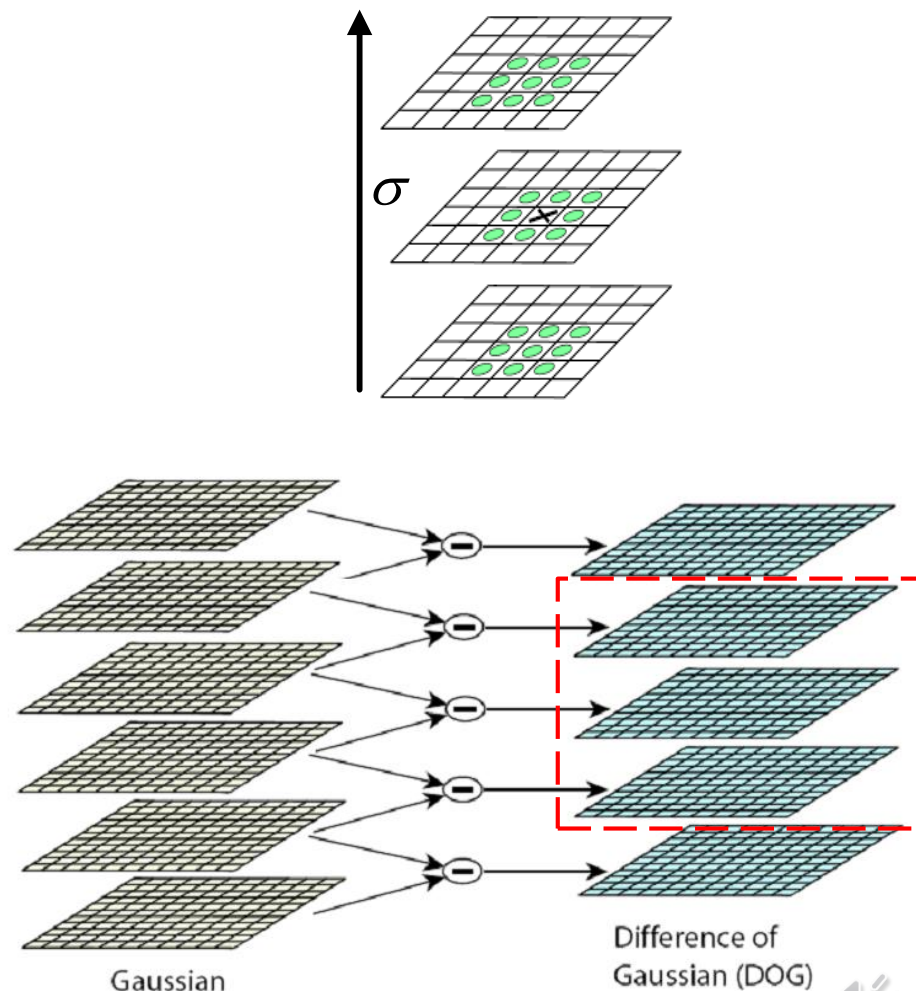
- doubleImageSize: 金字塔底层是否将原图像增倍，默认为true
- nOctaves: 图像金字塔组数，与图像大小有关

```
cvRound(log( (double)std::min( base.cols, base.rows ) ) / log(2.) - 2) - firstOctave
```



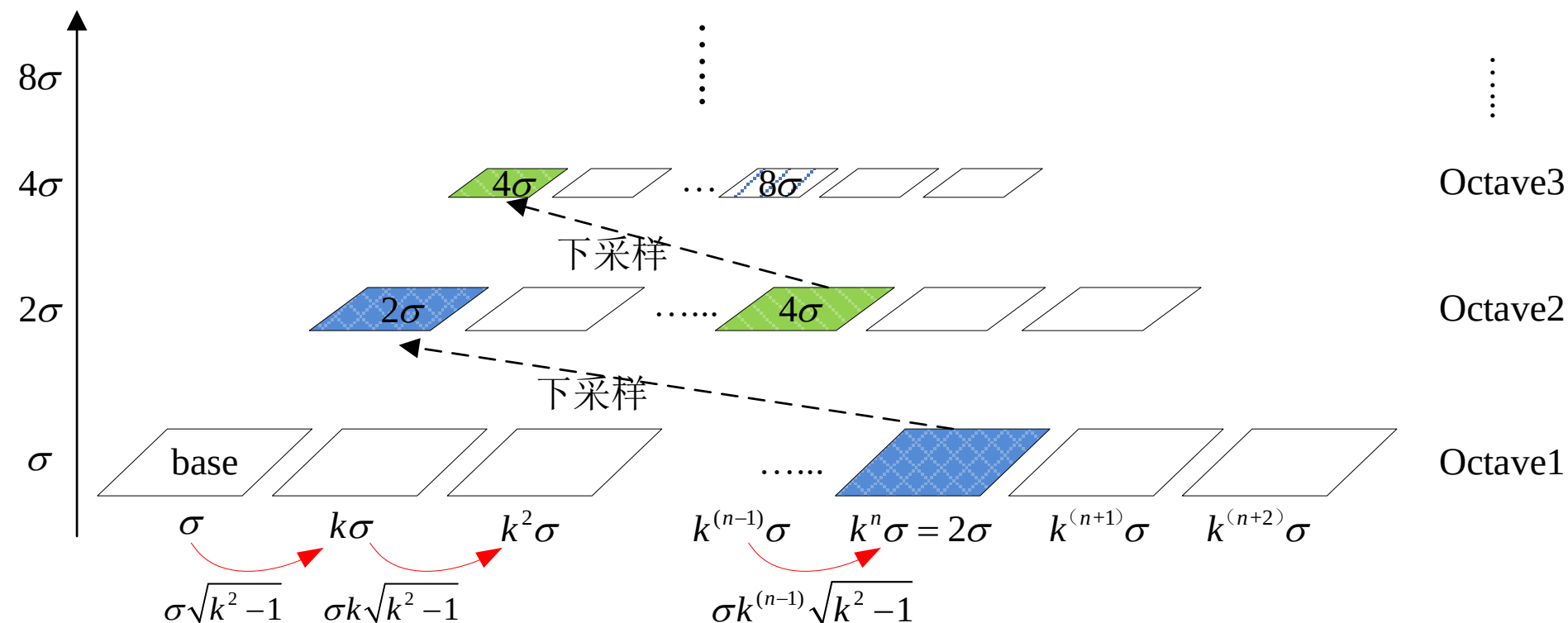
1) 构建图像金字塔

- nOctaves: 图像金字塔组数, 与图像大小有关
- nOctaveLayers (SIFT类构造函数参数): DoG金字塔中, 金字塔每组可检测特征的层数, 默认值为3
- 在高斯金字塔中, 每组图像数为 $nOctaveLayers + 3$, 高斯金字塔总图像数为:
 $nOctaves * (nOctaveLayers + 3)$
- 在DoG金字塔中, 每组图像数为 $nOctaveLayers + 2$, DoG金字塔总图像数为:
 $nOctaves * (nOctaveLayers + 2)$



1) 构建图像金字塔

➤ 高斯金字塔 $L(x, y, \sigma) = I(x, y) * g(x, y, \sigma)$ $g(x, y, \sigma) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$

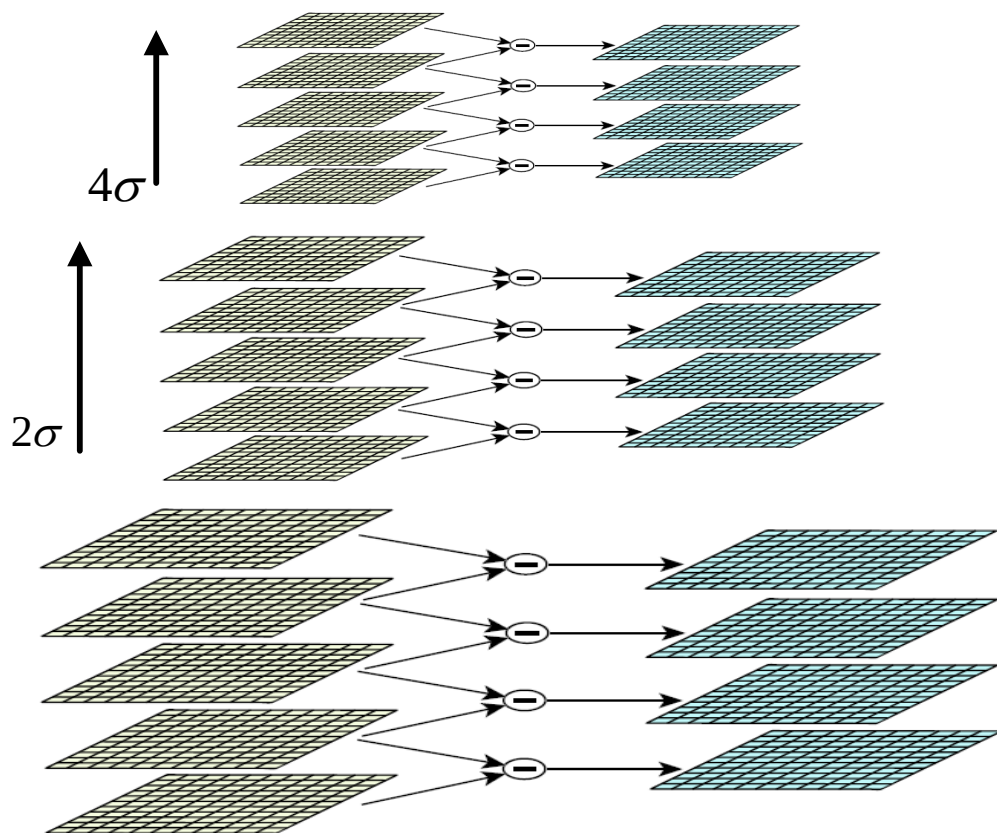


$$k = n_{OctaveLayers} \sqrt{2}$$

1) 构建图像金字塔

➤ DoG金字塔

$$\begin{aligned} D(x, y, \sigma) &= [G(x, y, k\sigma) - G(x, y, \sigma)] * I(x, y) \\ &= L(x, y, k\sigma) - L(x, y, \sigma) \end{aligned}$$



➤ 图像金字塔的两种采样方式

- 金字塔每组间：
图像空间域采样
- 金字塔同组各层间：
尺度域采样

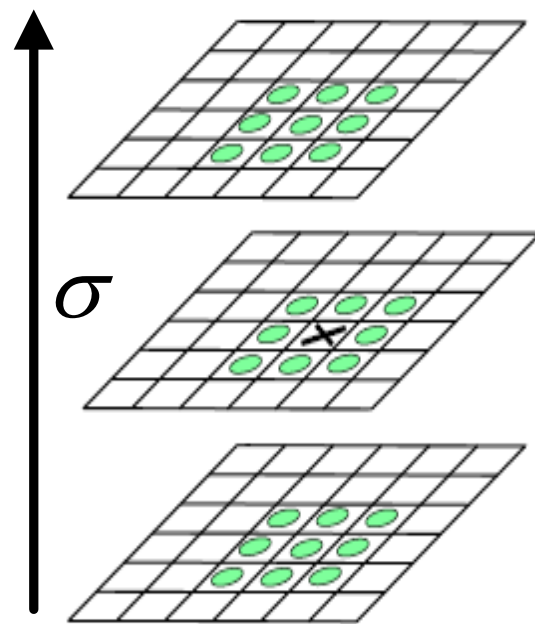
2) 检测极值点

➤ 非极大值抑制检测极值点

- 在DoG金字塔中，将当前点与立方体周围的26个点进行比较，检测局部极大/极小值

```
void SIFT::findScaleSpaceExtrema(  
    const vector<Mat>& gauss_pyr,  
    const vector<Mat>& dog_pyr,  
    vector<KeyPoint>& keypoints ) const
```

```
const Mat& img = dog_pyr[idx];  
const Mat& prev = dog_pyr[idx-1];  
const Mat& next = dog_pyr[idx+1];
```



3) 精确定位特征点

- 3.1) 精确定位极值点
- 3.2) 去除不稳定极值点
- 3.3) 去除边缘响应过大的极值点

```
static int adjustLocalExtrema( const vector<Mat>& dog_pyr, KeyPoint& kpt, int octv,  
                             int& layer, int& r, int& c, int nOctaveLayers,  
                             float contrastThreshold, float edgeThreshold, float sigma )
```

- dog_pyr: DoG金字塔
- Kpt: 保留特征点信息的数据结构
- octv, layer, r, c: 特征点所在的金字塔组数和层数、像素位置
- contrastThreshold: 去除不稳定极值点的相关阈值常量
- edgeThreshold: 去除边缘响应过大的极值点的相关阈值常量

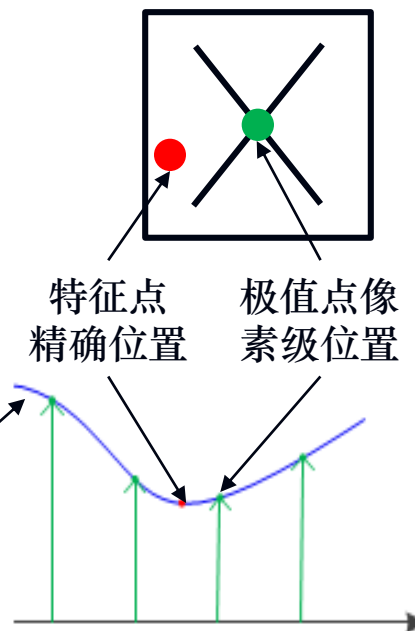
3.1) 精确定位特征点

- 通过采样极值点的三维二次函数，确定其精确位置
- $D(x, y, \sigma)$ 表示 σ 尺度下 (x, y) 点的高斯差分尺度空间值
- 设采样极值点在原点， $X = (x, y, \sigma)^T$ 表示精确定位值，其高斯差分尺度空间值（二阶泰勒级数展开）：

$$D(\mathbf{X}) = D + \frac{\partial D^T}{\partial \mathbf{X}} \mathbf{X} + \frac{1}{2} \mathbf{X}^T \frac{\partial^2 D}{\partial \mathbf{X}^2} \mathbf{X}$$

- 令 $D(\mathbf{X})$ 一阶导数为零，获取精确极值点

$$\frac{\partial D(\mathbf{X})}{\partial \mathbf{X}} = \frac{\partial D}{\partial \mathbf{X}} + \frac{\partial^2 D}{\partial \mathbf{X}^2} \hat{\mathbf{X}} = 0 \quad \hat{\mathbf{X}} = -\frac{\partial^2 D^{-1}}{\partial \mathbf{X}^2} \frac{\partial D}{\partial \mathbf{X}}$$



3.1) 精确定位极值点

➤ 线性方程求解：用差分近似偏导数

$$\hat{\mathbf{X}} = -\frac{\partial^2 D^{-1}}{\partial \mathbf{X}^2} \frac{\partial D}{\partial \mathbf{X}} \longrightarrow \begin{bmatrix} \frac{\partial^2 D}{\partial x^2} & \frac{\partial^2 D}{\partial xy} & \frac{\partial^2 D}{\partial x\sigma} \\ \frac{\partial^2 D}{\partial yx} & \frac{\partial^2 D}{\partial y^2} & \frac{\partial^2 D}{\partial y\sigma} \\ \frac{\partial^2 D}{\partial \sigma x} & \frac{\partial^2 D}{\partial \sigma y} & \frac{\partial^2 D}{\partial \sigma^2} \end{bmatrix} \begin{bmatrix} x \\ y \\ \sigma \end{bmatrix} = -\begin{bmatrix} \frac{\partial D}{\partial x} \\ \frac{\partial D}{\partial y} \\ \frac{\partial D}{\partial \sigma} \end{bmatrix}$$

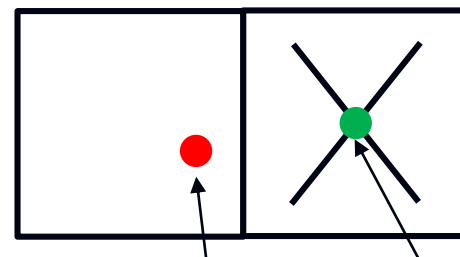
```
Vec3f dD((img.at<sift_wt>(r, c+1) - img.at<sift_wt>(r, c-1))*deriv_scale,
          (img.at<sift_wt>(r+1, c) - img.at<sift_wt>(r-1, c))*deriv_scale,
          (next.at<sift_wt>(r, c) - prev.at<sift_wt>(r, c))*deriv_scale);
```

```
Matx33f H(dxx, dxy, dxs,
          dxy, dyy, dys,
          dxs, dys, dss);
```

```
Vec3f X = H.solve(dD, DECOMP_LU);
```

3.1) 精确定位极值点

- 如果X在任何方向上大于0.5，表示极值点更接近另一个采样点，在该点重复计算



特征点
精确位置 极值点像
素级位置

```
for( ; i < SIFT_MAX_INTERP_STEPS; i++ )
```

```
xi = -X[2];
xr = -X[1];
xc = -X[0];
```

```
if( std::abs(xi) < 0.5f && std::abs(xr) < 0.5f && std::abs(xc) < 0.5f )
    break;
```

3) 精确定位特征点

- 3.2) 去除不稳定极值点：抛弃 $|D(\hat{\mathbf{X}})|$ 过小的点

$$D(\mathbf{X}) = D + \frac{\partial D^T}{\partial \mathbf{X}} \mathbf{X} + \frac{1}{2} \mathbf{X}^T \frac{\partial^2 D}{\partial \mathbf{X}^2} \mathbf{X}$$

$$\hat{\mathbf{X}} = -\frac{\partial^2 D^{-1}}{\partial \mathbf{X}^2} \frac{\partial D}{\partial \mathbf{X}} \quad \longrightarrow \quad D(\hat{\mathbf{X}}) = D + \frac{1}{2} \frac{\partial D^T}{\partial \mathbf{X}} \hat{\mathbf{X}}$$

- 3.3) 通过主曲率分析去除边缘响应过大的极值点

- 计算差分图像D的Hessian矩阵，保留满足条件极值点

$$\mathbf{H} = \begin{bmatrix} D_{xx} & D_{xy} \\ D_{xy} & D_{yy} \end{bmatrix} \quad \frac{\text{trace}(\mathbf{H})^2}{\det(\mathbf{H})} < \frac{(r+1)^2}{r} \quad r = 10$$

```
static int adjustLocalExtrema( const vector<Mat>& dog_pyr, KeyPoint& kpt, int octv,
                             int& layer, int& r, int& c, int nOctaveLayers,
                             float contrastThreshold, float edgeThreshold, float sigma)
```



3) 精确定位特征点

- 3.2) 去除不稳定极值点：抛弃 $|D(\hat{X})|$ 过小的点

$$D(\mathbf{X}) = D + \frac{\partial D^T}{\partial \mathbf{X}} \mathbf{X} + \frac{1}{2} \mathbf{X}^T \frac{\partial^2 D}{\partial \mathbf{X}^2} \mathbf{X}$$

$$\hat{\mathbf{X}} = -\frac{\partial^2 D^{-1}}{\partial \mathbf{X}^2} \frac{\partial D}{\partial \mathbf{X}} \quad \longrightarrow \quad D(\hat{\mathbf{X}}) = D + \frac{1}{2} \frac{\partial D^T}{\partial \mathbf{X}} \hat{\mathbf{X}}$$

- 3.3) 通过主曲率分析去除边缘响应过大的极值点

```
kpt.pt.x = (c + xc) * (1 << octv); // 特征点坐标
kpt.pt.y = (r + xr) * (1 << octv);
kpt.octave = octv + (layer << 8) + (cvRound((xi + 0.5)*255) << 16);
kpt.size = sigma*powf(2.f, (layer + xi) / nOctaveLayers)*(1 << octv)*2;=10
kpt.response = std::abs(contr);
```

极值点

```
static int adjustLocalExtrema( const vector<Mat>& dog_pyr, KeyPoint& kpt, int octv,
                               int& layer, int& r, int& c, int nOctaveLayers,
                               float contrastThreshold, float edgeThreshold, float sigma
```

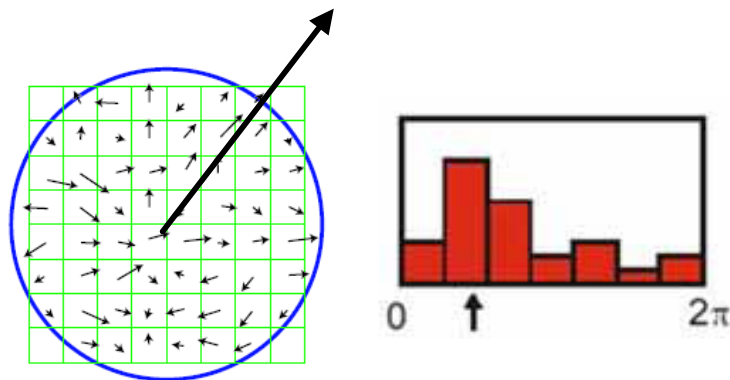


4) 设定特征点主方向

➤ 在特定像素点周围计算梯度方向直方图

```
static float calcOrientationHist( const Mat& img, Point pt, int radius,
                                float sigma, float* hist, int n )
```

- radius: 计算梯度方向直方图范围，和特征点半径有关
- hist: 梯度方向直方图
- n: 直方图bin数
- 返回值: 直方图最大值



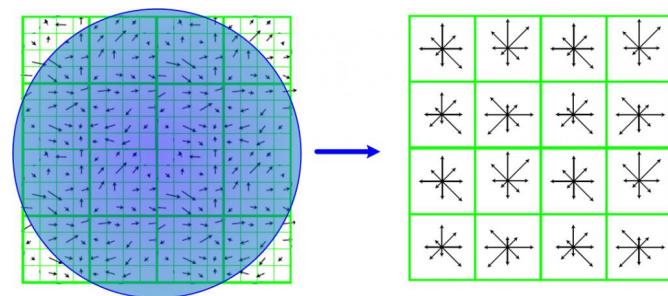
➤ 通过邻域梯度方向直方图确定特征的方向参数

- 特征点的主方向: 直方图的峰值
- 特征点的辅方向: 直方图取值超过主峰值能量的80%

5) 生成特征描述子

```
static void calcDescriptors(const vector<Mat>& gpyr, const vector<KeyPoint>& keypoints,
                          Mat& descriptors, int nOctaveLayers, int firstOctave )
,
```

- gpyr: 高斯金字塔
- keypoints: 完成检测的特征点向量
- descriptors: 需要计算的特征点描述子矩阵
- nOctaveLayers: 金字塔每组有效层数
- firstOctave: 金字塔底层编号



- 将坐标轴方向旋转为特征点的方向
- 以特征点为中心取窗口，通过高斯加权增强特征点附近像素梯度方向信息的贡献
- 将窗口分为 4×4 的小块，每块中计算梯度方向直方图（取8个方向），计算梯度方向累加值，形成种子点

5) 生成特征描述子

```
static void calcDescriptors(const vector<Mat>& gpyr, const vector<KeyPoint>& keypoints,  
                           Mat& descriptors, int nOctaveLayers, int firstOctave )  
,
```

- gpyr: 高斯金字塔
- keypoints: 完成检测的特征点向量
- descriptors: 需要计算的特征点描述子矩阵
- nOctaveLayers: 金字塔每组有效层数
- firstOctave: 金字塔底层编号

```
static void calcSIFTDescriptor( const Mat& img, Point2f ptf, float ori, float scl,  
                                int d, int n, float* dst )  
,
```

- img: 特征点所在图像
- ptf, ori, scl: 特征的像素级位置、方向、半径参数
- d, n: d=4,n=8
- dst: 特征点描述子, 维数: d*d*n

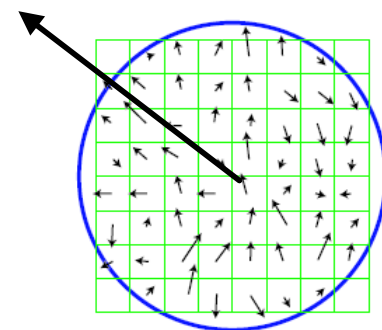
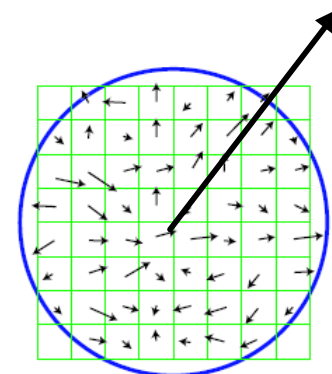
5) 生成特征描述子

- 特征点的旋转不变性
 - 当图像发生旋转时，特征点的主方向相对于像素的梯度方向不变
 - 将坐标轴方向旋转为特征点的方向

```
float cos_t = cosf(ori*(float)(CV_PI/180));
float sin_t = sinf(ori*(float)(CV_PI/180));
```

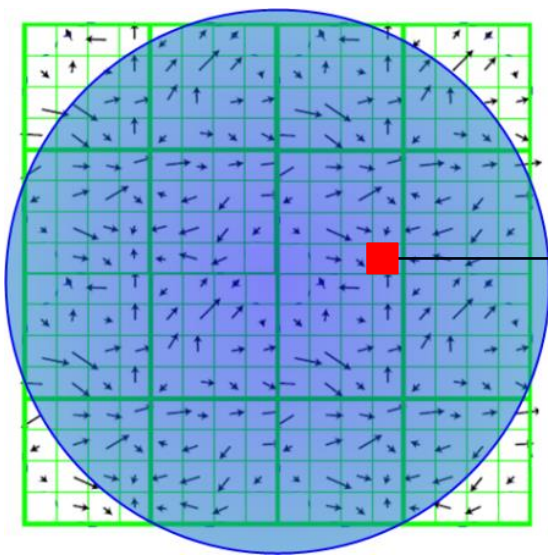
// 坐标旋转，从而保证特征的旋转不变性

```
float c_rot = j * cos_t - i * sin_t;
float r_rot = j * sin_t + i * cos_t;
```



5) 生成特征描述子

- 对每个特征点，构成 $4*4*8=128$ 维特征向量
- 对描述子的进一步处理
 - 去除光照影响：将特征向量的长度归一化



```
// 一个位置的梯度幅值和方向，作用于8个位置
int idx = ((r0+1)*(d+2) + c0+1)*(n+2) + o0;
hist[idx] += v_rco000;
hist[idx+1] += v_rco001;
hist[idx+(n+2)] += v_rco010;
hist[idx+(n+3)] += v_rco011;
hist[idx+(d+2)*(n+2)] += v_rco100;
hist[idx+(d+2)*(n+2)+1] += v_rco101;
hist[idx+(d+3)*(n+2)] += v_rco110;
hist[idx+(d+3)*(n+2)+1] += v_rco111;
```

总结



SIFTDetect ()

1. createInitialImage ()
 2. SIFT::buildGaussianPyramid ()
 3. SIFT::buildDoGPyramid ()
 4. SIFT::findScaleSpaceExtrema
 5. {
 6. for each layer, row, column
 7. find local extrema with pixel accuracy
 8. adjustLocalExtrema ()
 9. if isLocalExtrema
 10. calcOrientationHist ()
 11. }
 12. calcDescriptors ()
- 1) 构建图像金字塔
- 2) 检测极值点
- 3) 精确定位特征点
- 4) 设定特征点主方向
- 5) 生成特征描述子

特征匹配

- 建立图像的特征点数据库
 - 每个特征点的数据结构包括：位置坐标、尺度、方向、特征向量
- 为新图像的每个特征点在数据库中逐个匹配
 - 根据特征向量的欧氏距离在数据库中寻找其最近邻和次近邻特征点
 - 若(最近邻距离/次近邻距离)小于某一阈值，则特征匹配成功

```
void SIFT::matchAndDraw(const Mat& img1, const vector<KeyPoint>& keypoints1,  
    const Mat& img2, const vector<KeyPoint>& keypoints2,  
    Mat& outImg, Mat& outImg1, Mat& outImg2, bool ifDrawPoints) const
```

期待你的完成~~

The End



机器人与信息自动化研究所

Institute of Robotics & Automatic Information System



南開大學
Nankai University

